

Assurance Of Learning Report



Database Optimization and Design for Richard's Hotel Management

Written by:

Adhika Gunawan	2802438205
Daniel Sebastian Winata	2802392652
Hans Richard Djoj Santoso	2802392942
Marcell Hermawan Kristianto	2802392766
Nathanael Moses	2802400250
Tokesi Lukynawa	2802394525
Wilson Pranajaya Tungga	2802388075

Database Technology

Group 10

Class : LB09

Odd Semester 2025/2026

Table Of Contents

Table Of Contents	i
Table of Figures	iv
Table of Tables.....	vi
Chapter 1 Case Description.....	1
Chapter 2 Sample Data.....	2
A. Data Understanding.....	2
1. Transaction & Booking Information.....	2
2. Customer Information	2
3. Staff Information	2
4. Room & Pricing Information	3
5. Financial & Payment Information.....	3
B. Sample Data	4
Chapter 3 Anomalies and Dependencies.....	6
A. Data Problem.....	6
1. Data Redundancy	6
2. Data Inconsistencies.....	8
3. Presence of Derived Attributes	10
4. Ambiguous Data Context.....	11
5. Merged Cells	11
B. Functional Dependencies	12
1. Transaction Information.....	12
2. Customer and Staff Information	12
3. Room Inventory Dependencies	13
4. Payment Information.....	13
C. Anomalies	14
1. Insertion Anomaly.....	14
2. Update Anomaly	14

3. Delete Anomaly	15
Chapter 4 Implementation of Normalization	16
A. From UNF to 1NF	16
1. Ensuring Attribute Atomicity.....	16
2. Introducing a Simple Primary Key for Row Uniqueness	17
3. Identifying Derived Attributes and Unclear Context.....	18
4. Inconsistent Value in Data	18
5. Merged Cells	21
B. From 1NF to 2NF	22
1. Forming the MsTransaction Table.....	22
2. Forming the TransactionHeader Table.....	22
3. Normalizing Customer Attributes.....	23
4. Normalizing Staff Attributes.....	24
5. Normalizing Room Attributes.....	24
C. From 2NF to 3NF.....	25
1. Normalizing Payment Information into MsPaymentType.....	25
2. Normalizing Room Type Information into MsRoomDetail.....	26
Chapter 5 Entity Relationship Diagram	27
A. Tables Description	27
1. MsCustomer	27
2. MsPaymentType	28
3. MsRoom.....	28
4. MsRoomDetail	29
5. MsStaff.....	30
6. MsTransaction.....	31
7. TransactionHeader	32
B. Relationships Between Tables	34
1. MsCustomer to TransactionHeader (1-Many)	34
2. MsStaff to TransactionHeader (1-Many)	35
3. MsRoom to TransactionHeader (1-Many)	35
4. MsTransaction to TransactionHeader (1-1 or 1-Many).....	36

5.	MsPaymentType to MsTransaction (1–Many)	36
6.	MsRoomDetail to MsRoom (1–Many)	37
C.	Entity Relationship Diagram.....	38
D.	Conclusion	39
Chapter 6 Case Scenario And SQL File.....		40
A.	Case Scenario	40
1.	Case 1 : Calculating Total Revenue by Payment Type.....	40
2.	Case 2 : Identifying Staff Performance	41
3.	Case 3 : Checking Long Stay Customers	42
B.	SQL File	43

Table of Figures

Figure 2.1 Sample Data.....	4
Figure 2.2 Sample Data (Continued)	5
Figure 3.1 Repetitive Storage of 'Daniel Hayes' Customer Profile.....	6
Figure 3.2 Repetitive Storage of 'Matthew Ross' Staff Profile.....	7
Figure 3.3 Room and Pricing Data Redundancy.....	7
Figure 3.4 Inconsistent Capitalization in Customer Names.....	8
Figure 3.5 Inconsistent Phone Number Formatting	8
Figure 3.6 gmail Inconsistencies.....	9
Figure 3.7 Mail.com Inconsistencies	9
Figure 3.8 Asterisk Inconsistencies.....	9
Figure 3.9 Underscore Inconsistencies	9
Figure 3.10 Daniel Hayes's Conflicting Age.....	9
Figure 3.11 Derived Column Time Spent.....	10
Figure 3.12 Derived Column Total	10
Figure 3.13 Room Price	11
Figure 3.14 Payment Fee.....	11
Figure 3.15 Merged Cell in Customer Data	11
Figure 3.16 Transaction Attributes Dependent on TransactionID	12
Figure 3.17 Customer Attributes Dependent on their IDs	12
Figure 3.18 Staff Attributes Dependent on their respective IDs	12
Figure 3.19 Room Transitive Dependency	13
Figure 3.20 Payment Attributes Dependent on PaymentTypeID.....	13
Figure 3.21 Insertion Anomaly: Adding 'Penthouse' Room Type	14
Figure 3.22 Conflicting Phone Numbers for Same Customer	14
Figure 3.23 Unintended Loss of Staff Data	15
Figure 4.1 Violation of Attribute Atomicity	16
Figure 4.2 Normalized Customer Data Structure.....	17
Figure 4.3 TransactionId Primary Key.....	17
Figure 4.4 Derived Attributes	18

Figure 4.5 Unclear Context Price.....	18
Figure 4.6 Ryan and James Inconsistent Name	18
Figure 4.7 Standardized Customer Data Value.....	19
Figure 4.8 Daniel's Conflicting Age.....	19
Figure 4.9 Standardized Customer Data Values	20
Figure 4.10 Daniel's Email Inconcistencies.....	20
Figure 4.11 Standardized Customer Email	21
Figure 4.12 Merged Cell in Customer Data	21
Figure 4.13 Elimination of Merged Cells	21
Figure 4.14 Formation of MsTransaction Table	22
Figure 4.15 Formation of TransactionHeader Table.....	23
Figure 4.16 Formation of MsCustomer Table.....	24
Figure 4.17 Formation of MsRoom Table	24
Figure 4.18 Formation of Room Table	25
Figure 4.19 Formation of MsPaymentType Table.....	25
Figure 4.20 Formation of MsRoomDetail Table.....	26
Figure 5.1 ERD	38
Figure 6.1 SQL Query for Revenue Analysis Result :.....	40
Figure 6.2 Revenue Analysis Query Output	40
Figure 6.3 SQL Query for Staff Performance.....	41
Figure 6.4 Staff Performance Query Output.....	41
Figure 6.5 SQL Query for Long Stay Guests.....	42
Figure 6.6 Long Stay Guest Query Output	42

Table of Tables

Table 5.1 Attribute Descriptions for MsCustomer Table	27
Table 5.2 Key Constraints and Validation Rules for MsCustomer Table	27
Table 5.3 Attribute Descriptions for MsPaymentType Table.....	28
Table 5.4 Key Constraints and Validation Rules for MsPaymentType Table.....	28
Table 5.5 Attribute Descriptions for MsRoom Table	29
Table 5.6 Attribute Descriptions for MsRoom Table	29
Table 5.7 Attribute Descriptions for MsRoomDetail Table.....	29
Table 5.8 Key Constraints and Validation Rules for MsRoomDetail Tables	30
Table 5.9 Attribute Descriptions for MsStaff Table	30
Table 5.10 Key Constraints and Validation Rules for MsStaff Table	31
Table 5.11 Attribute Descriptions for MsTransaction Table	31
Table 5.12 Key Constraints and Validation Rules for MsTransaction Table	32
Table 5.13 Attribute Descriptions for TransactionHeader Table.....	32
Table 5.14 Key Constraints and Validation Rules for TransactionHeader Table.	33

Chapter 1

Case Description

The "Richard's Hotel" management system must process highly interconnected data streams, including customer bookings, room inventory, staff assignments, and detailed payment transactions. A non-centralized or unnormalized approach, such as using a single spreadsheet, introduces severe operational inefficiencies, including massive data redundancy and critical update, insertion, and deletion anomalies. This makes it impossible to reliably update a customer's details, add a new staff member without a transaction, or delete a booking without losing other valuable data. To address these challenges, this project involves designing and implementing a centralized, fully normalized relational database. This system ensures data integrity, eliminates redundancy, and provides a scalable foundation for managing all hotel operations, enabling efficient data retrieval through structured queries.

Chapter 2

Sample Data

A. Data Understanding

1. Transaction & Booking Information

This section details the booking information found in the raw dataset. The `TransactionID` (e.g., 'TR001') serves as the identifier for every booking line. The `TransactionDate` records when the booking was placed. `CheckInDateTime` and `CheckOutDateTime` specify the Duration of the stay. From these timestamps, a `Time Spent` value (e.g., 96) is derived and physically stored in the column, representing the Total duration.

2. Customer Information

In the Unnormalized Form (UNF), customer data is not stored atomically. Instead, a single composite column named `Customer` aggregates multiple pieces of information into one cell. This column contains the `CustomerName`, `CustomerAge`, `CustomerEmail`, and `CustomerTelp` separated by delimiters. Furthermore, this column utilizes merged cells to visually group multiple transaction rows associated with the same customer. While visually convenient, this merged structure creates incomplete tuples where underlying rows technically contain null values, complicating data processing.

3. Staff Information

Staff information, stored in `MsStaff`, identifies the employee who handled or recorded the transaction. The `StaffID` (e.g., 'ST001') uniquely identifies each staff member. `StaffName` specifies their name, while `StaffEmail` and `StaffTelp` provide essential contact information. This data clarifies who managed the transaction on behalf of the hotel.

4. Room & Pricing Information

Room inventory data is presented as a set of flat columns within the dataset. `RoomTypeID` acts as the category code, while `RoomType` (e.g., 'Suite') and `RoomPrice` describe the category and its nightly cost. The `RoomID` identifies the physical room, and `RoomAvailability` tracks its status. Notably, the `RoomPrice` column lists raw numeric values without explicitly stating the currency context (e.g., USD).

5. Financial & Payment Information

Financial attributes are recorded directly in the transaction rows alongside other data. `PaymentTypeID` and `PaymentTypeName` (e.g., 'Credit Card') identify the method used, while `PaymentFee` records the service charge. The dataset also includes a `Total` column, which is a derived attribute calculated by summing the `RoomPrice` and `PaymentFee`. This pre calculated value is stored directly in the dataset rather than being computed on demand, which introduces redundancy

B. Sample Data

TransactionID	TransactionDate	PaymentTypeName	PaymentFee	RoomID	RoomAvailability	RoomType	RoomPrice	CheckInDateTime	CheckOutDateTime
TR001	01/09/2025	Qris	0	RM001	Maintenance	Suite	900000	01/09/2025 02:00	05/09/2025 02:00
TR002	02/09/2025	Cash	0	RM012	Available	Double	500000	02/09/2025 01:00	06/09/2025 01:00
TR003	03/09/2025	Credit Card	1500	RM010	Occupied	Double	500000	03/09/2025 04:00	06/09/2025 04:00
TR004	04/09/2025	Debit Card	1000	RM016	Occupied	Suite	900000	04/09/2025 03:00	06/09/2025 03:00
TR005	05/09/2025	Qris	0	RM013	Occupied	Suite	900000	05/09/2025 01:00	11/09/2025 01:00
TR006	06/09/2025	Credit Card	1500	RM001	Maintenance	Suite	900000	06/09/2025 02:00	07/09/2025 02:00
TR007	07/09/2025	E-Wallet	500	RM016	Occupied	Suite	900000	07/09/2025 00:00	08/09/2025 00:00
TR008	08/09/2025	Credit Card	1500	RM016	Occupied	Suite	900000	08/09/2025 00:00	09/09/2025 00:00
TR009	09/09/2025	Qris	0	RM012	Available	Double	500000	09/09/2025 03:00	12/09/2025 03:00
TR010	10/09/2025	Credit Card	1500	RM011	Maintenance	Suite	900000	10/09/2025 00:00	15/09/2025 00:00
TR011	11/09/2025	Cash	0	RM005	Maintenance	Single	350000	11/09/2025 00:00	15/09/2025 00:00
TR012	12/09/2025	E-Wallet	500	RM011	Maintenance	Suite	900000	12/09/2025 01:00	16/09/2025 01:00
TR013	13/09/2025	Cash	0	RM013	Occupied	Suite	900000	13/09/2025 03:00	16/09/2025 03:00
TR014	14/09/2025	Debit Card	1000	RM003	Available	Deluxe	1200000	14/09/2025 01:00	17/09/2025 01:00
TR015	15/09/2025	Cash	0	RM015	Maintenance	Suite	900000	15/09/2025 04:00	17/09/2025 04:00
TR016	01/09/2025	Qris	0	RM002	Maintenance	Suite	900000	01/09/2025 02:00	05/09/2025 02:00
TR017	02/09/2025	Cash	0	RM004	Available	Deluxe	1200000	02/09/2025 01:00	06/09/2025 01:00
TR018	03/09/2025	Credit Card	1500	RM006	Maintenance	Single	350000	03/09/2025 04:00	06/09/2025 04:00
TR019	04/09/2025	Debit Card	1000	RM007	Available	Suite	900000	04/09/2025 03:00	06/09/2025 03:00
TR020	05/09/2025	Qris	0	RM008	Available	Single	350000	05/09/2025 01:00	11/09/2025 01:00
TR021	06/09/2025	Credit Card	1500	RM009	Occupied	Single	350000	06/09/2025 02:00	07/09/2025 02:00
TR022	07/09/2025	E-Wallet	500	RM014	Available	Presidential Suite	2400000	07/09/2025 00:00	08/09/2025 00:00
TR023	08/09/2025	Credit Card	1500	RM017	Occupied	Suite	900000	08/09/2025 00:00	09/09/2025 00:00
TR024	09/09/2025	Qris	0	RM018	Available	Deluxe	1200000	09/09/2025 03:00	12/09/2025 03:00
TR025	10/09/2025	Credit Card	1500	RM019	Occupied	Single	350000	10/09/2025 00:00	15/09/2025 00:00
TR026	11/09/2025	Cash	0	RM020	Occupied	Presidential Suite	2400000	11/09/2025 00:00	15/09/2025 00:00

Figure 2.1 Sample Data

CustomerID	Customer	StaffID	StaffName	StaffAge	StaffEmail	StaffTelp	Total	Time Spent
CU002	ryan coleman, 22, ryan.coleman@gmail.com, 8732826597	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	900000	96
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 879-057-4982	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	500000	96
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	501500	72
CU009	Chloe Reed, 45, chloe.reed@gmail.com, 8888338341	ST008	David Bennett	37	david.bennett@gmail.com	8691714489	901000	48
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982	ST008	David Bennett	37	david.bennett@gmail.com	8691714489	900000	144
CU007	Mia Simmons, 54, mia.simmons@gmail.com, 8439136046	ST004	Benjamin Brooks	51	benjamin.brooks@gmail.com	8469199586	901500	24
CU003	Sophia Ramirez, 35, sophia.ramirez@gmail.com, 8770727390	ST007	Lily Morgan	31	lily.morgan@gmail.com	8543524722	900500	24
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	901500	24
CU001	Laura Bennett, 49, laura_bennett@gmail.com, 8509734187	ST008	David Bennett	37	david.bennett@gmail.com	8691714489	500000	72
CU004	james Mitchell, 29, james.mitchell@gmail.com, 8197978717	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	901500	120
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	350000	96
CU010	Isabella Rogers, 27, isabella.rogers@gmail.com, 8520452472	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	900500	96
CU006	Lucas Morgan, 38, lucas.morgan@gamil.com, 8851878659	ST006	Samuel Ward	48	samuel.ward@gmail.com	8974153389	900000	72
CU003	Sophia Ramirez, 35, sophia.ramirez@gmail.com, 8770727390	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	1201000	72
CU006	Lucas Morgan, 38, lucas.morgan@gmail.com, 8851878659	ST003	Ella Patterson	54	ella.patterson@gmail.com	8679643587	900000	48
CU002	Ryan Coleman, 22, ryan.coleman@gmail.com, 8732826597	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	900000	96
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	1200000	96
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	351500	72
CU009	Chloe Reed, 45, chloe.reed@gmail.com, 8888338341	ST008	David Bennett	37	david.bennett@gmail.com	8691714489	901000	48
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982	ST008	David Bennett	37	david.bennett@gmail.com	8691714489	350000	144
CU007	Mia Simmons, 54, mia.simmons@gmail.com, 8439136046	ST004	Benjamin Brooks	51	benjamin.brooks@gmail.com	8469199586	351500	24
CU003	Sophia Ramirez, 35, sophia.ramirez@gmail.com, 877-072-7390	ST007	Lily Morgan	31	lily.morgan@gmail.com	8543524722	2400500	24
CU010	Daniel Hayes, 30, daniel.hayes@gmail.com, 8790574982	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	901500	24
CU001	Laura Bennett, 49, laura.bennett@gmail.com, 8509734187	ST008	David Bennett	37	david.bennett@gmail.com	8691714489	1200000	72
CU004	James Mitchell, 29, james.mitchell@mail.com, 8197978717	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	351500	120
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	2400000	96

Figure 2.2 Sample Data (Continued)

Chapter 3

Anomalies and Dependencies

A. Data Problem

1. Data Redundancy

The unnormalized dataset suffers from excessive repetition. Key information regarding Customers (Name, Email, Phone), Staff, and Room details is duplicated for every single transaction row. For example, the details for customer "Daniel Hayes" are repeated identically across more than ten separate rows. This redundancy wastes storage space and makes the data difficult to scan or manage efficiently.

a. Customer Data Redundancy

TransactionID	TransactionDate	CustomerID	CustomerName	CustomerAge	CustomerEmail	CustomerTelp
TR002	02/09/2025	CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982
TR003	03/09/2025	CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982
TR008	08/09/2025	CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982
TR020	05/09/2025	CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982
TR021	06/09/2025	CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982

Figure 3.1 Repetitive Storage of 'Daniel Hayes' Customer Profile

This redundancy occurs when patient identity information such as CustomerName, CustomerEmail, CustomerAge, and CustomerTelp is repeatedly recorded in every transaction involving the same guest. Because the exact same profile data is copied across multiple rows for every booking, any change to an attribute (for example, a new phone number for "Daniel Hayes") leads to immediate inconsistencies if it is not manually updated in all related historical rows.

b. Staff Data Redundancy

TransactionID	StaffID	StaffName	StaffAge	StaffEmail	StaffTelp
TR001	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
TR002	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
TR003	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
TR008	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
TR011	ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026

Figure 3.2 Repetitive Storage of 'Matthew Ross' Staff Profile

This redundancy appears when employee information, including StaffName, StaffEmail, and StaffTelp, is repeated in every transaction handled by that staff member. Since a single staff member manages many bookings over time, their personal details are duplicated hundreds of times. When a staff member's data changes, all rows containing their information must be updated, drastically increasing the risk of update anomalies and consuming unnecessary storage space.

c. Room and Pricing Data Redundancy

TransactionID	RoomID	RoomTypeID	RoomType	RoomPrice	RoomAvailability
TR001	RM001	RT003	Suite	900000	Maintenance
TR006	RM001	RT003	Suite	900000	Maintenance
TR011	RM011	RT003	Suite	900000	Maintenance
TR013	RM013	RT003	Suite	900000	Occupied
TR016	RM003	RT003	Suite	900000	Occupied

Figure 3.3 Room and Pricing Data Redundancy

This redundancy occurs when the RoomType, RoomPrice, and RoomAvailability status are repeatedly recorded in every transaction involving a specific room unit. Such repetition not only increases the volume of stored data but also creates critical risks for pricing management. If the price of a "Suite" changes, the system must update every future booking row individually, creating potential price discrepancies between identical room types.

2. Data Inconsistencies

a. Inconsistent Naming Convention

CustomerID	Customer
CU002	ryan coleman, 22, ryan.coleman@gmail.com, 8732826597
CU002	Ryan Coleman, 22, ryan.coleman@gmail.com, 8732826597
CU004	james Mitchell, 29, james.mitchell@gmail.com, 8197978717
CU004	James Mitchell, 29, james.mitchell@gmail.com, 8197978717

Figure 3.4 Inconsistent Capitalization in Customer Names

The customer name lacks a standard format for capitalization. For example, "ryan coleman" is written entirely in lowercase, while "james Mitchell" mixes uppercase and lowercase letters incorrectly. This inconsistency makes it difficult for the system to sort names or search for customers accurately.

b. Inconsistent Phone Number

CustomerID	Customer
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 879-057-4982
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982
CU007	Mia Simmons, 54, mia.simmons@gmail.com, 8439136046
CU003	Sophia Ramirez, 35, sophia.ramirez@gmail.com, 8770727390
CU003	Sophia Ramirez, 35, sophia.ramirez@gmail.com, 877-072-7390

Figure 3.5 Inconsistent Phone Number Formatting

The CustomerTelp field uses different formatting styles within the same column, some numbers are written as continuous strings, while others use different patterns. This lack of uniformity makes it impossible to automatically validate phone numbers or integrate them with messaging systems.

c. Invalid Email Syntax and Structure

CustomerID	Customer
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982
CU010	Daniel Hayes, 27, daniel.hayes*gmail.com, 8790574982

Figure 3.8 Asterisk Inconsistencies

CustomerID	Customer
CU004	James Mitchell, 29, james.mitchell@mail.com, 8197978717
CU004	james mitchell, 29, james.mitchell@gmail.com, 8197978717

Figure 3.7 Mail.com Inconsistencies

CustomerID	Customer
CU001	Laura Bennett, 49, laura.bennett@gmail.com, 8509734187
CU001	Laura Bennett, 49, laura_bennett@gmail.com, 8509734187

Figure 3.9 Underscore Inconsistencies

CustomerID	Customer
CU006	Lucas Morgan, 38, lucas.morgan@gmail.com, 8851878659
CU006	Lucas Morgan, 38, lucas.morgan@gamil.com, 8851878659

Figure 3.6 gamil Inconsistencies

The `CustomerEmail` column suffers from severe syntax violations and formatting inconsistencies. Critical errors include the use of invalid characters, such as an asterisk (*gmail.com) instead of the standard '@' symbol, and incomplete domains (@mail). Furthermore, the data contains typos in domain names (@gamil.com) and inconsistent separators, where some records deviate from the standard dot format by using underscores (_). These errors render the contact information invalid for actual communication and demonstrate a lack of input validation.

d. Conflicting Data Values

CustomerID	Customer
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982
CU010	Daniel Hayes, 30, daniel.hayes@gmail.com, 8790574982

Figure 3.10 Daniel Hayes's Conflicting Age

The dataset contains conflicting records for the same customer, proving that an update anomaly has occurred. Specifically, Daniel Hayes is recorded as 27 years old in most entries, but appears as 30 in

another. Since customer data is repeated across multiple rows, an update was not applied to all records. This creates a problem, making it impossible to know the customer's actual age.

3. Presence of Derived Attributes

RoomPrice	PaymentFee	Total
900000	0	900000
500000	0	500000
500000	1500	501500
900000	1000	901000
900000	0	900000
900000	1500	901500
900000	500	900500

Figure 3.12 Derived Column Total

CheckInDateTime	CheckOutDateTime	Time Spent
01/09/2025 02:00	05/09/2025 02:00	96
02/09/2025 01:00	06/09/2025 01:00	96
03/09/2025 04:00	06/09/2025 04:00	72
04/09/2025 03:00	06/09/2025 03:00	48
05/09/2025 01:00	11/09/2025 01:00	144
06/09/2025 02:00	07/09/2025 02:00	24
07/09/2025 00:00	08/09/2025 00:00	24

Figure 3.11 Derived Column Time Spent

The dataset contains columns that are not raw data but are calculated from other fields. Specifically, the `Time Spent` column is derived from the difference between `CheckOutDateTime` and `CheckInDateTime`, and the `Total` column is a calculation based on `RoomPrice` and `PaymentFee`. Storing these calculated values physically in the table creates a risk where the calculated result might not match the actual inputs if an update occurs. For example if a guest extends their stay, the stored `Time Spent` value becomes incorrect.

4. Ambiguous Data Context

PaymentFee
0
0
1500
1000
0
1500
500
1500

Figure 3.14 Payment Fee

RoomPrice
900000
500000
500000
900000
900000
900000
900000
900000

Figure 3.13 Room Price

Certain numeric values in the dataset lack clear descriptive units, leading to potential misinterpretation of critical data. A prime example is found in financial columns such as `RoomPrice` and `PaymentFee`, which list raw numbers without an explicit currency indicator (such as IDR or USD). Since these attributes are essential and will be retained in the final database structure, the absence of a defined monetary context creates ambiguity that could lead to significant errors during financial analysis or reporting.

5. Merged Cells

CustomerID	Customer	StaffID	StaffName	StaffAge	StaffEmail	StaffTelp	Total
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 879-057-4982	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	500000
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	501500
CU004	James Mitchell, 29, james.mitchell@gmail.com, 819797871	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	901500
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	350000
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	1200000
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	351500
CU004	James Mitchell, 29, james.mitchell@mail.com, 819797871	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	351500
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	2400000

Figure 3.15 Merged Cell in Customer Data

The dataset uses merged cells to visually group repeated records, such as `CustomerName` and `StaffName`. While this makes the data easier for humans to read, it creates a structural failure for database processing. Technically, only the top row contains the value, leaving all subsequent rows empty (NULL). This results in incomplete tuples, making it impossible to sort, filter, or query the data correctly without losing context.

B. Functional Dependencies

1. Transaction Information

TransactionID	TransactionDate	CheckInDateTime	CheckOutDateTime
TR001	01/09/2025	01/09/2025 02:00	05/09/2025 02:00
TR002	02/09/2025	02/09/2025 01:00	06/09/2025 01:00
TR003	03/09/2025	03/09/2025 04:00	06/09/2025 04:00

Figure 3.16 Transaction Attributes Dependent on TransactionID

The TransactionID serves as the primary key for all booking related schedules, defining the TransactionDate, CheckInDateTime, and CheckOutDateTime. In the unnormalized data, calculated columns like Time Spent and Total are actually derived from other variables (Duration of stay and combined costs) rather than depending directly on the key. Therefore, all temporal data regarding a specific booking relies entirely on the TransactionID for unique identification.

2. Customer and Staff Information

CustomerID	CustomerName	CustomerAge	CustomerEmail	CustomerTelp
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	8790574982

Figure 3.17 Customer Attributes Dependent on their IDs

For the customer entity, attributes such as CustomerName, CustomerAge, CustomerEmail, and CustomerTelp are functionally dependent on the customer, not the transaction itself. In the raw data, a transitive relationship exists where the transaction implies a customer, but the customer's personal details actually depend on CustomerID.

StaffID	StaffName	StaffAge	StaffEmail	StaffTelp
ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026

Figure 3.18 Staff Attributes Dependent on their respective IDs

Similarly, for the staff entity, attributes including `StaffName`, `StaffAge`, `StaffEmail`, and `StaffTelp` depend on the specific employee. To eliminate redundancy and inconsistencies, unique identifiers `CustomerID` and `StaffID` must be used to represent each person. This ensures that personal attributes depend directly on their respective keys rather than being repeated in every transaction record.

3. Room Inventory Dependencies

RoomID	RoomTypeID	RoomType	RoomPrice
RM001	RT003	Suite	900000
RM006	RT003	Suite	900000
RM011	RT003	Suite	900000

Figure 3.19 Room Transitive Dependency

Every physical unit is distinguished by a `RoomID`, which controls its `RoomAvailability` status and links it to a category via `RoomTypeID`. A transitive dependency is observed here although `RoomID` identifies the unit, the cost (`RoomPrice`) and category name (`RoomType`) are actually determined by the `RoomTypeID`. Consequently, the price depends on the classification of the room rather than the physical room itself, necessitating a separation to facilitate efficient pricing updates.

4. Payment Information

TransactionID	PaymentTypeID	PaymentTypeName	PaymentFee
TR003	PT002	Credit Card	1500
TR006	PT002	Credit Card	1500
TR008	PT002	Credit Card	1500

Figure 3.20 Payment Attributes Dependent on `PaymentTypeID`

Each payment method is identified by `PaymentTypeID`, which determines the specific attributes of the payment channel, namely `PaymentTypeName` and the associated `PaymentFee`. In the transaction context, the `TransactionID` determines which `PaymentTypeID` was used, but the details of the fee and the method

name are functionally dependent solely on the `PaymentTypeID`. This structure ensures that if a payment fee changes, it only needs to be updated in the payment master record, not in every individual transaction.

C. Anomalies

1. Insertion Anomaly

TransactionID	TransactionDate	CustomerName	StaffID	StaffName	StaffEmail	RoomTypeID	RoomType	RoomPrice
TR001	01/09/2025	Ryan Coleman	ST002	Matthew Ross	matthew.ross@gmail.com	RT003	Suite	900000
NULL	NULL	NULL	ST099	John Doe	john.doe@gmail.com	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	RT006	Penthouse	5000000

Figure 3.21 Insertion Anomaly: Adding 'Penthouse' Room Type

We cannot record new master data independently. For instance, a newly hired staff member cannot be registered into the database until they handle their first booking. Similarly, a new room category cannot be created and stored in the system until a guest actually books a room of that type. This prevents the hotel from maintaining a master list of employees or inventory independent of sales activity.

2. Update Anomaly

TransactionID	TransactionDate	CustomerName	CustomerEmail	CustomerTelp	StaffName
TR002	02/09/2025	Daniel Hayes	daniel.hayes@gmail.com	8790574982	Matthew Ross
TR003	03/09/2025	Daniel Hayes	daniel.hayes@gmail.com	8790574982	Matthew Ross
TR008	08/09/2025	Daniel Hayes	daniel.hayes@gmail.com	81299998888	Matthew Ross
TR020	05/09/2025	Daniel Hayes	daniel.hayes@gmail.com	8790574982	Grace Turner

Figure 3.22 Conflicting Phone Numbers for Same Customer

Updates are inefficient and error prone due to duplicated data. Since customer 'Daniel Hayes' appears in multiple rows, changing his phone number requires editing every single entry manually. If just one row is overlooked, the data becomes inconsistent. The same issue applies to updates for Room Prices or Staff Emails.

3. Delete Anomaly

TransactionID	TransactionDate	CustomerName	StaffID	StaffName	StaffEmail	StaffTelp
TR006	06/09/2025	Lucas Morgan	ST004	Benjamin Brooks	benjamin.brooks@gmail.com	8469199586
TR007	07/09/2025	Mia Simmons	ST007	Lily Morgan	lily.morgan@gmail.com	8543524722
TR008	08/09/2025	Daniel Hayes	ST002	Matthew Ross	matthew.ross@gmail.com	8891871026

Figure 3.23 Unintended Loss of Staff Data

Deleting a transaction results in the unintended loss of other important data. For example, staff member 'Lily Morgan' is currently linked to only one booking. If we delete that booking record, all her personal details will be permanently erased from the system. It will be like she never worked at the hotel at all.

Chapter 4

Implementation of Normalization

The implementation of this normalization process aims to transform the raw data (UNF) from the “Richard’s Hotel” system into a fully structured relational database schema up to the Third Normal Form (3NF). Each step is designed to systematically eliminate the issues identified in Chapter 3, including data redundancy, problematic functional dependencies, and various data anomalies.

A. From UNF to 1NF

1. Ensuring Attribute Atomicity

CustomerID	Customer
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 879-057-4982
CU004	james Mitchell, 29, james.mitchell@gmail.com, 8197978717
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982
CU004	James Mitchell, 29, james.mitchell@mail.com, 8197978717

Figure 4.1 Violation of Attribute Atomicity

The Unnormalized Form (UNF) table presented a significant violation of attribute atomicity, specifically within the Customer column. This column functioned as a composite attribute, containing four distinct pieces of information Name, Age, Email, and Phone Number crammed into a single cell and separated only by commas. This structure makes it impossible for the database to query specific details, such as sorting customers by age or filtering by email domain.

To achieve First Normal Form (1NF) the composite column was decomposed into its atomic parts. We split the original Customer column into four separate, distinct columns: CustomerName, CustomerAge, CustomerEmail, and CustomerTelp. This transformation ensures that every cell now contains only a single, indivisible value, allowing for accurate data processing and efficient indexing.

CustomerID	CustomerName	CustomerAge	CustomerEmail	CustomerTelp
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	08790574982
CU004	James Mitchell	29	james.mitchell@gmail.com	08197978717

Figure 4.2 Normalized Customer Data Structure

2. Introducing a Simple Primary Key for Row Uniqueness

Primary Key

TransactionID	TransactionDate	PaymentTypeName	PaymentFee	RoomID	RoomAvailability	RoomType	RoomPrice	CheckInDateTime	CheckOutDateTime
TR001	2025-09-01	Qris	0	RM001	Maintenance	Suite	900000	2025-09-01 02:00	2025-09-05 02:00
TR002	2025-09-02	Cash	0	RM012	Available	Double	500000	2025-09-02 01:00	2025-09-06 01:00
TR003	2025-09-03	Credit Card	1500	RM010	Occupied	Double	500000	2025-09-03 04:00	2025-09-06 04:00
TR004	2025-09-04	Debit Card	1000	RM016	Occupied	Suite	900000	2025-09-04 03:00	2025-09-06 03:00
TR005	2025-09-05	Qris	0	RM013	Occupied	Suite	900000	2025-09-05 01:00	2025-09-11 01:00
TR006	2025-09-06	Credit Card	1500	RM001	Maintenance	Suite	900000	2025-09-06 02:00	2025-09-07 02:00
TR007	2025-09-07	E-Wallet	500	RM016	Occupied	Suite	900000	2025-09-07 00:00	2025-09-08 00:00
TR008	2025-09-08	Credit Card	1500	RM016	Occupied	Suite	900000	2025-09-08 00:00	2025-09-09 00:00
TR009	2025-09-09	Qris	0	RM012	Available	Double	500000	2025-09-09 03:00	2025-09-12 03:00
TR010	2025-09-10	Credit Card	1500	RM011	Maintenance	Suite	900000	2025-09-10 00:00	2025-09-15 00:00

Figure 4.3 TransactionID Primary Key

Without a unique row identifier, a UNF structure may rely on implicit grouping that allows duplicate or ambiguous entries. To enforce uniqueness and deterministic identity, a simple primary key is introduced as identified in Chapter 2. The TransactionID attribute for example “TR001” is assigned as the Primary Key. This is very different from cases that require a composite key, because in this model TransactionID alone is sufficient to distinguish each booking row. This ensures that every transaction tuple is unique and that all operations on it remain unambiguous.

3. Identifying Derived Attributes and Unclear Context

Total	Time Spent
900000	96
500000	96
501500	72
901000	48
900000	144

Figure 4.4 Derived Attributes

RoomPrice
900000
500000
500000
900000
900000

Figure 4.5 Unclear Context Price

During the transition toward 1NF, we directly addressed the data quality issues discussed in Chapter 3. We identified that `Time Spent` and `Total` were derived attributes, calculated from duration and cost summations respectively. Storing these values violates the 1NF principle of atomicity and creates redundancy. Therefore, these calculated columns were removed from the table structure at this stage. We also noted the ambiguity regarding units (e.g., currency for `RoomPrice`), which will be resolved by explicit definition in the final Data Dictionary. The resulting 1NF table now consists only of raw, atomic facts.

4. Inconsistent Value in Data

a. Inconsistent in Customer Name

CustomerID	Customer
CU002	ryan coleman, 22, ryan.coleman@gmail.com, 8732826597
CU002	Ryan Coleman, 22, ryan.coleman@gmail.com, 8732826597
CU004	james Mitchell, 29, james.mitchell@gmail.com, 8197978717
CU004	James Mitchell, 29, james.mitchell@gmail.com, 8197978717

Figure 4.6 Ryan and James Inconsistent Name

The initial Unnormalized Form (UNF) table presented a significant data inconsistency issue within the `Customer` column, specifically affecting the `CustomerName`. In the raw data, for `CustomerID` CU002, the customer's name was recorded as "ryan coleman" in lowercase, while for the subsequent entry for the same `CustomerID`, it appeared as "Ryan Coleman" with proper capitalization. Similarly, `CustomerID` CU004 showed "james

mittell" and "James Mitchell". Although referring to the same individuals, these differing capitalizations create distinct text strings for a computer, hindering accurate data retrieval, grouping, and analysis.

When the data was transformed into First Normal Form (1NF) by splitting the composite Customer column, the inconsistency was addressed by standardizing the CustomerName to a consistent format, typically by applying proper case or a predetermined standard, as seen in the corrected table where both "Ryan Coleman" and "James Mitchell" are uniformly capitalized. This ensures that each unique customer is represented by a single, consistent name, facilitating reliable queries and reporting.

CustomerID	CustomerName	CustomerAge
CU002	Ryan Coleman	22
CU004	James Mitchell	29
CU004	James Mitchell	29
CU002	Ryan Coleman	22

Figure 4.7 Standardized Customer Data Value

b. Inconsistent in Customer Age

CustomerID	Customer
CU010	Daniel Hayes, 30, daniel.hayes@gmail.com, 8790574982
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982

Figure 4.8 Daniel's Conflicting Age

Another critical data inconsistency observed in the Unnormalized Form (UNF) table relates to the Customer Age within the composite Customer column. For CustomerID CU010, the raw data displayed conflicting age values: "Daniel Hayes, 30, ..." in one record and "Daniel Hayes, 27, ..." in another. This direct contradiction for the same customer identifier poses a significant problem for data integrity, as it becomes ambiguous which age is correct. Such

discrepancies can arise from data entry errors or outdated information. Upon conversion to First Normal Form (1NF), where the Customer column was decomposed, the CustomerAge attribute was isolated. The corrected table reflects a singular, resolved age of "27" for CustomerID CU010.

CustomerID	CustomerName	CustomerAge	CustomerEmail
CU010	Daniel Hayes	27	daniel.hayes@gmail.com

Figure 4.9 Standardized Customer Data Values

c. Inconsistent in Customer Email

CustomerID	Customer
CU010	Daniel Hayes, 30, daniel.hayes@gmail.com, 8790574982
CU010	Daniel Hayes, 27, daniel.hayes*gmail.com, 8790574982

Figure 4.10 Daniel's Email Inconcistencies

The Unnormalized Form (UNF) also demonstrated an inconsistency within the Customer Email component of the composite Customer column. For CustomerID CU010, the raw data showed "daniel.hayes@gmail.com" in one entry but "daniel.hayes*gmail.com" (with an asterisk) in another. This type of inconsistency, often stemming from typos or special characters being mistakenly included, makes it impossible for a system to correctly identify identical email addresses or to use them for communication. Databases treat "daniel.hayes@gmail.com" and "daniel.hayes*gmail.com" as entirely different addresses. When the data was normalized to First Normal Form (1NF), the CustomerEmail attribute was extracted as a distinct column. In the new 1NF table, the email for CustomerID CU010 is now consistently "daniel.hayes@gmail.com". This standardization is crucial for ensuring valid communication, unique identification, and accurate filtering based on email addresses.

CustomerID	CustomerName	CustomerAge	CustomerEmail
CU010	Daniel Hayes	27	daniel.hayes@gmail.com

Figure 4.11 Standardized Customer Email

5. Merged Cells

CustomerID	Customer	StaffID	StaffName	StaffAge	StaffEmail	StaffTelp	Total
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 879-057-4982	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	500000
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	501500
CU004	James Mitchell, 29, james.mitchell@gmail.com, 819797871	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	901500
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	350000
CU010	Daniel Hayes, 27, daniel.hayes@gmail.com, 8790574982	ST001	Grace Turner	51	grace.turner@gmail.com	8186704863	1200000
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	351500
CU004	James Mitchell, 29, james.mitchell@mail.com, 819797871	ST005	Ava Cooper	58	ava.cooper@gmail.com	8321900348	351500
		ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026	2400000

Figure 4.12 Merged Cell in Customer Data

The presence of merged cells in the raw Unnormalized Form (UNF) data presents a critical structural violation for database integrity. In the initial spreadsheet view, certain values span vertically across multiple rows to indicate that a single piece of information applies to a group of records. While visually tidy for human readers, this creates ambiguity for a database system, which cannot definitively determine to which specific row a merged value belongs, thereby violating the 1NF requirement that every cell must contain a single, atomic value for a specific record.

To transition this data into 1NF, as shown in the corrected table, these merged cells must be resolved. This is achieved by unmerging the cells and repeating the appropriate value into every individual cell for each distinct row that was originally covered by the merge. This process ensures that every single row is a complete, standalone record with explicit values in every column, which is essential for accurate sorting, filtering, and querying.

CustomerID	CustomerName	CustomerAge	CustomerEmail	CustomerTelp	StaffID	StaffName	StaffAge	StaffEmail	StaffTelp
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	08790574982	ST001	Grace Turner	51	grace.turner@gmail.com	08186704863
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	08790574982	ST002	Matthew Ross	32	matthew.ross@gmail.com	08891871026
CU004	James Mitchell	29	james.mitchell@gmail.com	08197978717	ST005	Ava Cooper	58	ava.cooper@gmail.com	08321900348
CU004	James Mitchell	29	james.mitchell@gmail.com	08197978717	ST002	Matthew Ross	32	matthew.ross@gmail.com	08891871026
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	08790574982	ST001	Grace Turner	51	grace.turner@gmail.com	08186704863
CU010	Daniel Hayes	27	daniel.hayes@gmail.com	08790574982	ST002	Matthew Ross	32	matthew.ross@gmail.com	08891871026
CU004	James Mitchell	29	james.mitchell@gmail.com	08197978717	ST005	Ava Cooper	58	ava.cooper@gmail.com	08321900348
CU004	James Mitchell	29	james.mitchell@gmail.com	08197978717	ST002	Matthew Ross	32	matthew.ross@gmail.com	08891871026

Figure 4.13 Elimination of Merged Cells

B. From 1NF to 2NF

1. Forming the MsTransaction Table

In the 1NF structure, the attributes directly related to a transaction such as TransactionDate, CheckInDate, CheckOutDate, PaymentTypeName, and PaymentFee all depend entirely on TransactionID. However, these attributes are still mixed with information about the customer, staff, and room. To enforce the principles of 2NF, the group of transaction attributes is separated into a table named MsTransaction which uses TransactionID as the primary key. This table stores only the information that is fully functionally dependent on TransactionID, thereby removing redundancy that previously appeared when these values were presented together with customer, staff, and room attributes.

TransactionID	TransactionDate	CheckInDateTime	CheckOutDateTime	PaymentTypeName	PaymentFee
TR001	2025-09-01	2025-09-01 02:00	2025-09-05 02:00	Qris	0
TR002	2025-09-02	2025-09-02 01:00	2025-09-06 01:00	Cash	0
TR003	2025-09-03	2025-09-03 04:00	2025-09-06 04:00	Credit Card	1500
TR004	2025-09-04	2025-09-04 03:00	2025-09-06 03:00	Debit Card	1000
TR005	2025-09-05	2025-09-05 01:00	2025-09-11 01:00	Qris	0
TR006	2025-09-06	2025-09-06 02:00	2025-09-07 02:00	Credit Card	1500
TR007	2025-09-07	2025-09-07 00:00	2025-09-08 00:00	E-Wallet	500
TR008	2025-09-08	2025-09-08 00:00	2025-09-09 00:00	Credit Card	1500
TR009	2025-09-09	2025-09-09 03:00	2025-09-12 03:00	Qris	0
TR010	2025-09-10	2025-09-10 00:00	2025-09-15 00:00	Credit Card	1500
TR011	2025-09-11	2025-09-11 00:00	2025-09-15 00:00	Cash	0
TR012	2025-09-12	2025-09-12 01:00	2025-09-16 01:00	E-Wallet	500
TR013	2025-09-13	2025-09-13 03:00	2025-09-16 03:00	Cash	0

Figure 4.14 Formation of MsTransaction Table

2. Forming the TransactionHeader Table

Although MsTransaction successfully isolates the transaction attributes, a linking table is needed to ensure that each transaction can be related to the three main entities: Customer, Staff, and Room. To meet this requirement, a table named TransactionHeader is created. It functions as a linking table with the following attributes:

- HeaderID as the primary key

- TransactionID as a foreign key referencing MsTransaction
- CustomerID as a foreign key referencing MsCustomer
- StaffID as a foreign key referencing MsStaff
- RoomID as a foreign key referencing Room

TransactionHeader does not store descriptive attributes. It only stores references to the related entities. This makes the database structure more modular and fully compliant with the principles of 2NF because no non key attribute depends on an inappropriate key.

HeaderID	TransactionID	RoomID	StaffID	CustomerID
TH001	TR001	RM001	ST002	CU002
TH002	TR002	RM012	ST001	CU010
TH003	TR003	RM010	ST002	CU010
TH004	TR004	RM016	ST008	CU009
TH005	TR005	RM013	ST008	CU010

Figure 4.15 Formation of TransactionHeader Table

3. Normalizing Customer Attributes

In the 1NF table, the attributes CustomerName, CustomerAge, CustomerEmail, and CustomerTelp appear repeatedly in every transaction row. These attributes do not depend on TransactionID but instead depend functionally on CustomerID. To ensure that every non key attribute depends fully on the primary key of its table, a table named MsCustomer is created with CustomerID as the primary key and the customer descriptive attributes as non key attributes. The transaction structure now stores only CustomerID as a foreign key through TransactionHeader. This separation removes redundancy and eliminates the risk of update anomalies in customer data.

CustomerID	CustomerName	CustomerAge	CustomerEmail	CustomerTelp
CU001	Laura Bennett	49	laura.bennett@gmail.com	08509734187
CU002	Ryan Coleman	22	ryan.coleman@gmail.com	08732826597
CU003	Sophia Ramirez	35	sophia.ramirez@gmail.com	08770727390
CU004	James Mitchell	29	james.mitchell@gmail.com	08197978717
CU005	Isabella Rogers	27	isabella.rogers@gmail.com	08520452472

Figure 4.16 Formation of MsCustomer Table

4. Normalizing Staff Attributes

Staff attributes such as `StaffName`, `StaffAge`, `StaffEmail`, and `StaffTelp` also do not depend on `TransactionID` but depend entirely on `StaffID`. To satisfy the rules of 2NF, these attributes are moved into a table named `MsStaff`, which uses `StaffID` as its primary key. With this structure, any change to staff data is applied to a single row in the `MsStaff` table rather than to many transaction rows stored in the 1NF structure.

StaffID	StaffName	StaffAge	StaffEmail	StaffTelp
ST001	Grace Turner	51	grace.turner@gmail.com	8186704863
ST002	Matthew Ross	32	matthew.ross@gmail.com	8891871026
ST003	Ella Patterson	54	ella.patterson@gmail.com	8679643587
ST004	Benjamin Brooks	51	benjamin.brooks@gmail.co	08469199586

Figure 4.17 Formation of MsRoom Table

5. Normalizing Room Attributes

In the 1NF structure, room attributes such as `RoomType`, `RoomAvailability`, and `RoomPrice` appear repeatedly even though they do not depend on `TransactionID`. All of these attributes instead depend fully on `RoomID`. To enforce 2NF, these attributes are moved into a table named `Room` with `RoomID` as the primary key. As a result, the transaction structure no longer stores room details and instead stores only `RoomID` through `TransactionHeader`. This separation prevents inconsistencies in room prices, availability, and room types across transactions.

RoomID	RoomAvailability	RoomType	RoomPrice
RM001	Maintenance	Suite	900000
RM002	Maintenance	Suite	900000
RM003	Available	Deluxe	1200000
RM004	Available	Presidential	2400000

Figure 4.18 Formation of Room Table

C. From 2NF to 3NF

1. Normalizing Payment Information into MsPaymentType

In the 2NF structure, the attributes `PaymentTypeName` and `PaymentFee` are still attached to the `MsTransaction` table even though these attributes do not depend on `TransactionID` as its primary key. These attributes are fully determined by `PaymentTypeID` which serves as the identifier of the payment method.

To eliminate this transitive dependency, a table named `MsPaymentType` is created. It uses `PaymentTypeID` as the primary key and stores the descriptive payment attributes `PaymentTypeName` and `PaymentFee`. The `MsTransaction` table then stores only `PaymentTypeID` as a foreign key.

With this separation:

- Information about payment methods is managed centrally.
- Changes to the admin fee or the payment method name are made only once in the `MsPaymentType` table.
- `MsTransaction` contains clean transaction facts without derived attributes.

PaymentTypeID	PaymentTypeName	PaymentFee
PT001	Cash	0
PT002	Credit Card	1500
PT003	Debit Card	1000
PT004	E-Wallet	500
PT005	Qris	0

Figure 4.19 Formation of MsPaymentType Table

2. Normalizing Room Type Information into MsRoomDetail

In the 2NF structure, attributes such as RoomType and RoomPrice are still stored in the Room table even though these attributes do not directly depend on RoomID. In the “Richard’s Hotel” system, the attributes that determine the type and price of a room are fully determined by RoomTypeID rather than by RoomID.

To correct this transitive dependency, a table named MsRoomDetail is created with RoomTypeID as the primary key and the descriptive attributes RoomType and RoomPrice. The Room table then stores only RoomTypeID as a foreign key.

With this separation:

- Room prices and room types are no longer attached to RoomID, which removes redundancy.
- Changes to room type or room price are made only once in the MsRoomDetail table.
- Each Room entity stores only its unique information such as RoomID and RoomAvailability without carrying attributes that do not belong to it.

RoomTypeID	RoomType	RoomPrice
RT001	Single	350000
RT002	Double	500000
RT003	Suite	900000
RT004	Deluxe	1200000
RT005	Presidential Suite	2400000

Figure 4.20 Formation of MsRoomDetail Table

Chapter 5

Entity Relationship Diagram

A. Tables Description

1. MsCustomer

The table serves as a reference of customers of Richard's hotel. The columns are:

Table 5.1 Attribute Descriptions for MsCustomer Table

Column	Description
CustomerID	Unique identifier for each customer. Used to reference the customer in other tables.
CustomerName	Registered name for identification.
CustomerAge	Age for display, searching, and grouping.
CustomerEmail	Registered email address of customer.
CustomerTelp	Direct contact number for communication.

The keys and constraints are:

Table 5.2 Key Constraints and Validation Rules for MsCustomer Table

Constraint	Column	Description
Primary Key	CustomerID	Ensuring each customer entry is uniquely identifiable
Unique	CustomerEmail	Unique constraint that enforces full email uniqueness.
Check	CustomerID REGEXP '^CU[0-9]{3}\$'	Validates CustomerID format.

Not Null	(CustomerID, CustomerName, CustomerEmail)	Each customer must contain at least a customer ID, name, and email.
----------	---	---

2. MsPaymentType

The table represents the customer's payment details. The columns are:

Table 5.3 Attribute Descriptions for MsPaymentType Table

Column	Description
PaymentTypeID	Unique identifier for each payment type.
PaymentTypeName	Registered official name for a payment type.
PaymentFee	The amount of fee for every payment type.

The keys and constraints are:

Table 5.4 Key Constraints and Validation Rules for MsPaymentType Table

Constraint	Column	Description
Primary Key	PaymentTypeID	Ensures every payment type has its own unique identifier for simple identification.
Check	PaymentTypeID REGEXP '^PT[0-9]{3}\$'	Validates the PaymentTypeID format.
Not Null	(PaymentTypeID, PaymentTypeName)	Each type of payment must contain at least the payment type ID and name.

3. MsRoom

The table represents rooms of the hotel and if they are available. The columns are:

Table 5.5 Attribute Descriptions for MsRoom Table

Column	Description
RoomID	Unique identifier for each room.
RoomTypeID	Unique identifier for the type of room.
RoomAvailability	Status of the room's availability. (Maintenance,Available,Occupied)

The keys and constraints are:

Table 5.6 Attribute Descriptions for MsRoom Table

Constraint	Column	Description
Primary Key	RoomID	Ensuring every room is uniquely identifiable.
Foreign Key	RoomTypeID	References the RoomTypeID from the MsRoomDetail table.
Check	(RoomID, RoomTypeID) RoomID REGEXP '^RM[0-9]{3}\$' RoomTypeID REGEXP '^RT[0-9]{3}\$'	Validates the RoomID and the RoomTypeID formats.
Not Null	(RoomID, RoomTypeID)	Each room must contain at least the room ID and type ID.

4. MsRoomDetail

The table represents every room type's details and prices. The columns are:

Table 5.7 Attribute Descriptions for MsRoomDetail Table

Column	Description
--------	-------------

RoomTypeID	Unique identifier for the type of room.
RoomType	The type of room associated with the room type ID.
RoomPrice	The price of every room type. (In USD)

The keys and constraints are:

Table 5.8 Key Constraints and Validation Rules for MsRoomDetail Tables

Constraint	Column	Description
Primary Key	RoomTypeID	Ensuring every room type is unique.
Check	RoomTypeID REGEXP ‘^RT[0-9]{3}\$’	Validates the RoomTypeID format.
Not Null	All Columns	Each room details must contain full room type and price info.

5. MsStaff

The table represents staff members of the hotel and their information. The columns are:

Table 5.9 Attribute Descriptions for MsStaff Table

Column	Description
StaffID	Unique identifier for each staff member. Used to reference the staff in other tables.
StaffName	Registered name of every staff for identification
StaffAge	Age of a staff member
StaffEmail	Staff’s email address.
StaffTelp	Direct contact number for communication.

The keys and constraints are:

Table 5.10 Key Constraints and Validation Rules for MsStaff Table

Constraint	Column	Description
Primary Key	StaffID	Ensuring each staff entry is uniquely identifiable
Unique	StaffEmail	Unique constraint that enforces full email uniqueness.
Check	StaffID REGEXP '^ST[0-9]{3}\$'	Validates StaffID format.
Not Null	(StaffID, StaffName, StaffEmail)	Each staff must contain at least a staff ID, name, and email.

6. MsTransaction

The table represents all transactions recorded in Richard's Hotel.

Table 5.11 Attribute Descriptions for MsTransaction Table

Column	Description
TransactionID	Unique identifier for every transaction made. Used for searching, grouping, and identifying.
PaymentTypeID	Unique identifier for what payment type the transaction has.
TransactionDate	The recorded date when the transaction took place.
CheckInDateTime	Exact date and time of the check in of the transaction.
CheckOutDateTime	Exact date and time of the check out of

	the transaction.
--	------------------

The keys and constraints are:

Table 5.12 Key Constraints and Validation Rules for MsTransaction Table

Constraint	Column	Description
Primary Key	TransactionID	Ensuring each transaction is uniquely identifiable.
Foreign Key	PaymentTypeID	References PaymentTypeID from MsPaymentType.
Check	(TransactionID, PaymentTypeID) TransactionID REGEXP ‘^TR[0-9]{3}\$’ PaymentTypeID REGEXP ‘^PT[0-9]{3}\$’	Validates the TransactionID and PaymentTypeID formats.
Not Null	All Columns	Each transaction entry must contain full transaction information.

7. TransactionHeader

The table represents a header for Richard’s Hotel that shows all transactions with the customer, staff, and room ID. The columns are:

Table 5.13 Attribute Descriptions for TransactionHeader Table

Column	Description
HeaderID	Unique identifier for a header.
CustomerID	Unique identifier for a customer.
StaffID	Unique identifier for a staff.
RoomID	Unique identifier for a room.

TransactionID	Unique identifier for a transaction.
---------------	--------------------------------------

The keys and constraints are:

Table 5.14 Key Constraints and Validation Rules for TransactionHeader Table

Constraint	Column	Description
Primary Key	HeaderID	Ensures every header has a unique identifier.
Foreign Key	(CustomerID, StaffID, RoomID, TransactionID)	Ensures the IDs are referencing their respective variable types from previous tables. CustomerID referencing MsCustomer StaffID referencing MsStaff RoomID referencing MsRoom TransactionID referencing MsTransaction
Check	All Columns HeaderID REGEXP '^TH[0-9]{3}\$' CustomerID REGEXP '^CU[0-9]{3}\$' StaffID REGEXP '^ST[0-9]{3}\$' RoomID REGEXP '^RM[0-9]{3}\$'	Validates the HeaderID, CustomerID, StaffID, RoomID, and TransactionID formats.

	TransactionID REGEXP '^TR[0-9]{3}\$'	
Not Null	All Columns	Each header must contain full identifiers about the customer, staff, room, and transaction.

B. Relationships Between Tables

The database for Richard's Hotel is designed using the Crow's foot method, a fully normalized relational structure, where every table is connected through well defined primary and foreign key relationships. These relationships ensure data integrity, reduce redundancy, and allow efficient retrieval of interconnected booking, customer, room, and payment information. The main relationships are described below:

1. MsCustomer to TransactionHeader (1-Many)

A single customer may have multiple bookings over time.

Primary Key: MsCustomer.CustomerID

Foreign Key: TransactionHeader.CustomerID

This relationship ensures that every header entry references a valid existing customer, allowing the system to track who made each booking. This relationship is secured by ON UPDATE CASCADE to ensure data consistency, should the customer's master code ever change, the corresponding link in all transaction records is automatically corrected. Conversely, ON DELETE RESTRICT is applied to strictly prevent the permanent deletion of any customer profile that still holds active or completed bookings, thereby safeguarding the hotel's critical historical and audit data.

2. **MsStaff to TransactionHeader (1–Many)**

Each transaction header is associated with exactly one staff member who handled the booking.

Primary Key: MsStaff.StaffID

Foreign Key: TransactionHeader.StaffID

This relationship links booking records to specific employees responsible for processing them. This relationship uses ON UPDATE CASCADE to propagate changes instantly, ensuring that if a staff member's ID code is altered in the master table, the change is automatically reflected across all historical transaction records. Meanwhile, ON DELETE RESTRICT is enforced to prohibit the deletion of a staff member's record as long as they are associated with any recorded transaction, maintaining the integrity of the audit trail.

3. **MsRoom to TransactionHeader (1–Many)**

Rooms are assigned to booking headers, and a room may appear in many headers over time.

Primary Key: MsRoom.RoomID

Foreign Key: TransactionHeader.RoomID

This connection allows retrieval of historical booking activity for each room and supports room availability management. This relationship uses ON UPDATE CASCADE to guarantee that any update to the **RoomID** in the master table is instantly updated in all corresponding **TransactionHeader** records, maintaining the record's integrity. ON DELETE RESTRICT is strictly applied to prevent a specific physical room from being removed from the inventory if it has booking history still recorded in the **TransactionHeader** table.

4. **MsTransaction to TransactionHeader (1–1 or 1–Many)**

Each transaction header references a single transaction record, but depending on business rules, a transaction can be linked to one or multiple headers (e.g., grouped bookings).

Primary Key: MsTransaction.TransactionID

Foreign Key: TransactionHeader.TransactionID

This ensures that each header corresponds to a valid transaction that stores the financial and timing details. This relationship uses **ON UPDATE CASCADE** to ensure any update to the **TransactionID** is automatically reflected in the header, maintaining the record's integrity. **ON DELETE RESTRICT** is necessary to prevent the deletion of the core financial and temporal details (**MsTransaction**) if that record is still referenced by a header, ensuring that every header always corresponds to a valid transaction record.

5. **MsPaymentType to MsTransaction (1–Many)**

A payment type can be used in multiple transactions, but each transaction uses exactly one payment type.

Primary Key: MsPaymentType.PaymentTypeID

Foreign Key: MsTransaction.PaymentTypeID

This structure allows easy updates of payment fee configurations without changing individual transaction records. This relationship uses **ON UPDATE CASCADE** to ensure flexible management of payment methods. If the `PaymentTypeID` is updated, all corresponding records in `MsTransaction` are automatically synchronized. **ON DELETE RESTRICT** prevents the removal of any payment type (e.g., 'Credit Card') if it has been used in existing transactions, preserving the validity of the historical financial records.

6. **MsRoomDetail to MsRoom (1–Many)**

Room types determine the characteristics and pricing for multiple physical rooms.

Primary Key: MsRoomDetail.RoomTypeID

Foreign Key: MsRoom.RoomTypeID

This ensures that each room inherits the correct pricing, category, and type details based on its room type. This relationship uses ON UPDATE CASCADE to ensure that any change to the master **RoomTypeID** is automatically applied to all physical rooms linked to that type, maintaining consistency across the inventory. ON DELETE RESTRICT prevents the deletion of a room type if there are still physical rooms classified under that type, which is critical for price and inventory management integrity.

C. Entity Relationship Diagram

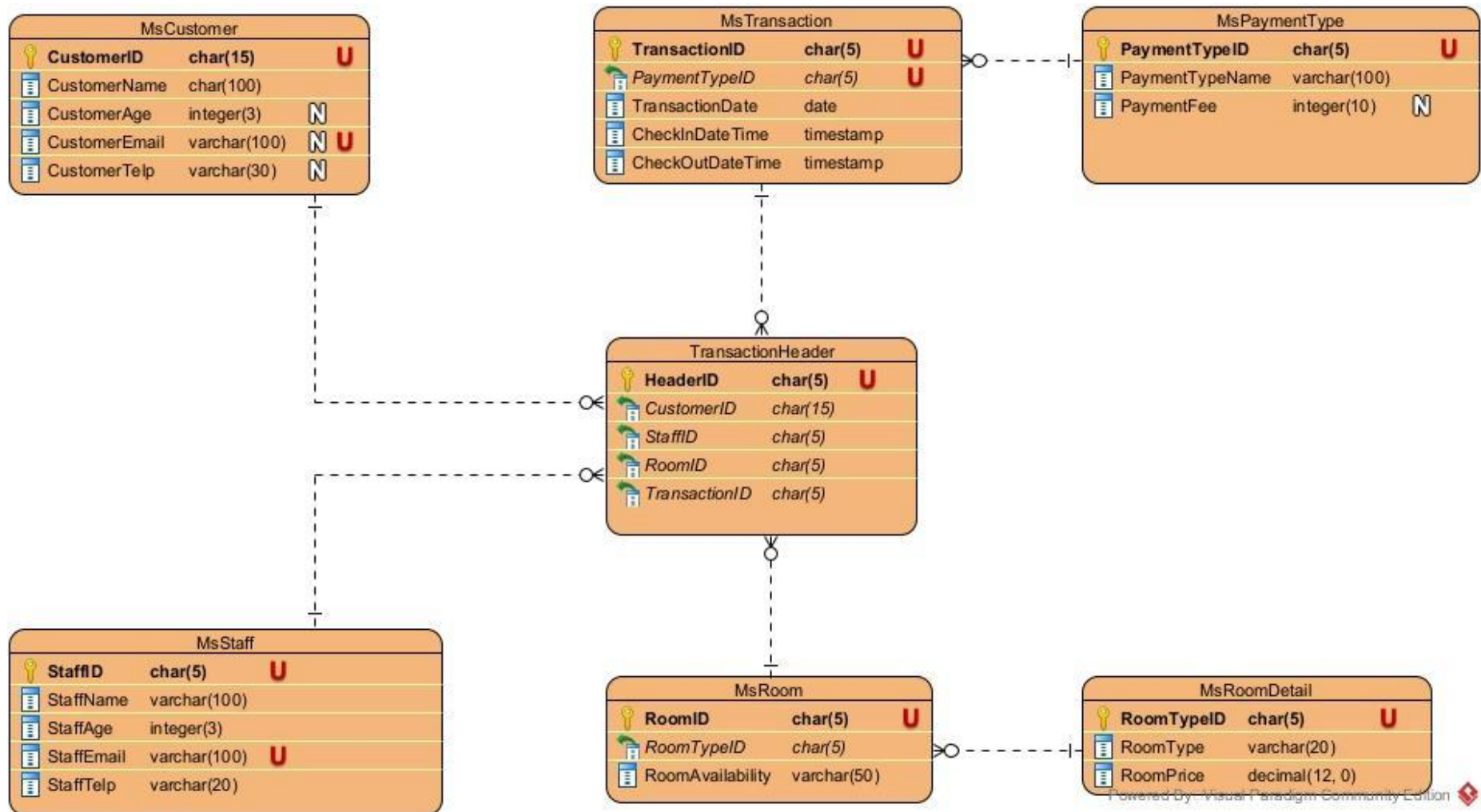


Figure 5.1 ERD

D. Conclusion

The final database design for Richard's Hotel establishes a fully normalized and logically sound structure essential for efficient hotel operations. This structure was developed through progressive normalization up to the Third Normal Form (3NF).

Key Outcomes of the Database Refinement :

- **Normalization Success:** All final tables comply with 3NF principles, meaning every piece of data is stored only once. This successfully eliminates all partial and transitive dependencies, achieving minimal data redundancy.
- **Referential Integrity:** Critical Foreign Key (FK) relationships are strictly implemented across all tables. This ensures that every booking is consistently and accurately linked to a valid guest, room, and staff member.
- **Structural Clarity:** The design separates entities into Master and Transaction tables. This separation enhances data maintainability, simplifies updates, and improves analytical traceability.
- **Scalability & Reliability:** The use of unique identifiers (`CustomerID`, `StaffID`) and associative tables (`TransactionHeader`) provides a strong, scalable foundation. The system can efficiently manage high volumes of bookings and ensures that changes to master data are instantly cascaded across all records, guaranteeing system reliability.
- **Resolution of Anomalies:** The final structure completely resolves the data anomalies identified in the initial unnormalized form (UNF), such as the update, insertion, and deletion problems associated with repeating customer and staff details.

Overall, the implemented relational database provides a robust and reliable foundation that is well balanced for performance and scalability, making all hotel operational processes from booking management to financial recording efficient and reliable for future growth.

Chapter 6

Case Scenario And SQL File

A. Case Scenario

1. Case 1 : Calculating Total Revenue by Payment Type

The finance department wants to analyze which payment methods generate the most revenue. They need to group transactions by payment type and calculate the total amount paid, including fees.

Query :

```
SELECT
    pt.PaymentTypeName,
    SUM(
        (rd.RoomPrice
         * DATEDIFF(t.CheckOutDateTime, t.CheckInDateTime))
        + pt.PaymentFee
    ) AS TotalRevenue
FROM MsPaymentType pt
JOIN MsTransaction t ON pt.PaymentTypeID = t.PaymentTypeID
JOIN TransactionHeader th ON t.TransactionID = th.TransactionID
JOIN MsRoom r ON th.RoomID = r.RoomID
JOIN MsRoomDetail rd ON r.RoomTypeID = rd.RoomTypeID
GROUP BY pt.PaymentTypeName;
```

Figure 6.1 SQL Query for Revenue Analysis

Result :

PaymentTypeName	TotalRevenue
Cash	27100000
Credit Card	13962000
Debit Card	6103000
E-Wallet	6901500
Qris	19800000

Figure 6.2 Revenue Analysis Query Output

Discussion :

The results clearly highlight which payment channels generate the most revenue. This insight allows the finance team to optimize costs by negotiating better rates for high volume methods or incentivizing customers to use lower fee options like Cash or Qris. Conversely, methods with low usage can be evaluated for system accessibility or potential removal.

2. Case 2 : Identifying Staff Performance

Hotel managers want to evaluate staff productivity for monthly reviews. They want to know how many transactions each staff member handled within September.

Query :

```
SELECT
    s.StaffName,
    COUNT(th.TransactionID) AS TotalTransactions
FROM MsStaff s
JOIN TransactionHeader th ON s.StaffID = th.StaffID
JOIN MsTransaction t ON th.TransactionID = t.TransactionID
WHERE MONTH(t.TransactionDate) = 9
    AND YEAR(t.TransactionDate) = 2025
GROUP BY s.StaffName
ORDER BY TotalTransactions DESC
```

Figure 6.3 SQL Query for Staff Performance

Result :

StaffName	TotalTransactions ▾ 1
Matthew Ross	7
David Bennett	6
Lily Morgan	3
Grace Turner	3
Ava Cooper	3
Benjamin Brooks	2
Ella Patterson	1
Samuel Ward	1

Figure 6.4 Staff Performance Query Output

Discussion :

The data identifies Matthew Ross and David Bennett as top performers (7 and 6 transactions), indicating high efficiency. Conversely, Ella Patterson and Samuel Ward show minimal activity (1 each), potentially due to role assignments or fewer shifts. These insights allow management to optimize scheduling by prioritizing top performers for busy periods and identifying training needs for others.

3. Case 3 : Checking Long Stay Customers

Sometimes guests stay much longer than usual, and the hotel wants to monitor long-stay guests for operational purposes (cleaning schedules, billing adjustments). Find all transactions where the stay duration is longer than 5 days.

Query :

```
SELECT
    th.TransactionID,
    c.CustomerName,
    t.CheckInDateTime,
    t.CheckOutDateTime,
    DATEDIFF(t.CheckOutDateTime, t.CheckInDateTime) AS DurationDays
FROM TransactionHeader th
JOIN MsTransaction t ON th.TransactionID = t.TransactionID
JOIN MsCustomer c ON th.CustomerID = c.CustomerID
WHERE DATEDIFF(t.CheckOutDateTime, t.CheckInDateTime) > 5;
```

Figure 6.5 SQL Query for Long Stay Guests

Result :

TransactionID	CustomerName	CheckInDateTime	CheckOutDateTime	DurationDays
TR005	Daniel Hayes	2025-09-05 01:00:00	2025-09-11 01:00:00	6
TR020	Daniel Hayes	2025-09-05 01:00:00	2025-09-11 01:00:00	6

Figure 6.6 Long Stay Guest Query Output

Discussion :

The query identified guests with a 6 day duration, exceeding the 5 day threshold. This insight allows housekeeping to optimize cleaning schedules and resources. Additionally, management can target these high value customers with loyalty perks to encourage retention and improve the accuracy of room availability forecasting.

B. SQL File

The .sql file is attached in the same folder where this report file is.